

Core Python

Curriculum



Core Python

Module 1 : Introduction to Programming & Python

- What is Programming?
- Why Python?
- Python Applications
- Installing Python
- IDE Setup (VS Code)
- Running Python Programs
- Python Interactive Shell
- Python Syntax
- Indentation
- Comments
- Variables
- Naming Conventions
- Keywords
- Data Types
- Type Casting
- Input & Output

Module 2 : Lists

- What is a List?
- Creating Lists
- Accessing List Items
- Negative Indexing
- Slicing Lists
- Checking if Item Exists
- Updating List Items
- Adding Items
 - append()
 - insert()
 - extend()

- Removing Items
 - remove()
 - pop()
 - del
 - clear()
- Looping Through Lists
- Sorting Lists
- Copying Lists
- Joining Lists
- Nested Lists
- List Methods

Module 3 : Tuples

- What is a Tuple?
- Creating Tuples
- Accessing Tuple Items
- Negative Indexing
- Slicing Tuples
- Packing Tuples
- Unpacking Tuples
- Updating Tuples
- Looping Through Tuples
- Joining Tuples
- Tuple Methods

Module 4 : Sets

- What is a Set?
- Creating Sets
- Accessing Set Items
- Adding Items
- Updating Sets
- Removing Items
- Looping Through Sets
- Joining Sets
- Union

- Intersection
- Difference
- Symmetric Difference
- Set Methods

Module 5 : Dictionaries

- What is a Dictionary?
- Creating Dictionaries
- Accessing Values
- Accessing Keys
- Accessing Items
- Updating Values
- Adding Items
- Removing Items
- Dictionary Methods
- Looping Through Dictionaries
- Copying Dictionaries
- Nested Dictionaries

Module 6 : Decision Making

- Operators
 - Arithmetic Operators
 - Assignment Operators
 - Comparison Operators
 - Logical Operators
 - Identity Operators
 - Membership Operators
 - Bitwise Operators
- Boolean Values
- if Statement
- if...else
- elif
- Nested if
- Short-hand if
- Short-hand if...else

- Logical Operators in Conditions
- match-case

Module 7 : Loops

- while Loop
- for Loop
- range()
- Looping Through Strings
- Looping Through Lists
- Looping Through Tuples
- Looping Through Sets
- Looping Through Dictionaries
- Nested Loops
- break
- continue
- pass

Module 8 : Strings

- What is a String?
- Creating Strings
- Multiline Strings
- String Indexing
- Negative Indexing
- String Slicing
- String Methods
- String Formatting
- Escape Characters
- f-Strings
- String Concatenation

Module 9 : Functions

- What is a Function?
- Creating a Function
- Calling a Function

- Arguments
- Parameters vs Arguments
- Number of Arguments
- Positional Arguments
- Keyword Arguments
- Default Parameter Values
- Arbitrary Arguments (*args)
- Arbitrary Keyword Arguments (**kwargs)
- Positional-only Arguments
- Keyword-only Arguments
- Passing Lists as Arguments
- Passing Dictionaries as Arguments
- Return Values
- Multiple Return Values
- Variable Scope
- Local Variables
- Global Variables
- The pass Statement
- Recursive Functions
- Lambda Functions
- Anonymous Functions
- Built-in Functions
- User-defined Functions

Module 10 : Modules & Packages

- What is a Module?
- Creating Modules
- Importing Modules
- Import Specific Functions
- Module Aliasing
- Built-in Modules
- Creating Packages
- Installing Packages using pip
- Virtual Environment (Introduction)

- Common Modules
- math
- random
- datetime
- statistics
- os
- sys

Module 11 : Exception Handling

- What are Exceptions?
- try
- except
- else
- finally
- raise
- Custom Exceptions
- Multiple Exceptions
- Exception Objects

Module 12 : File Handling

- What are Files?
- File Modes
- Opening Files
- Reading Files
- Writing Files
- Appending Files
- Closing Files
- Working with Text Files
- Working with CSV Files
- Working with JSON Files

Module-wise Assessments & Challenges

To ensure effective learning, every module concludes with a set of:

- Practice Exercises
- Coding Challenges
- Real-world Programming Tasks
- Logic Building Problems
- Debugging Activities

These assessments are designed to evaluate understanding, reinforce concepts, and prepare students for subsequent modules.

Final Capstone Projects (Core Python Only)

These projects combine everything learned in Core Python.

1. Banking Management System

Uses

- Lists
- Dictionaries
- Functions
- File Handling
- Exception Handling

2. Student Management System

Uses

- CRUD
- CSV
- Search
- Reports

3. Hospital Management System

Uses

- Patient Records
- Doctors
- Billing

4. Library Management System

Uses

- Book Inventory
- Borrow/Return
- Fine Calculation

5. Inventory & Billing Software

Uses

- Products
- Billing
- Invoice
- Stock Management

Course Outcomes

Upon successful completion of this course, learners will be able to:

1. Understand programming fundamentals and apply logical thinking to solve computational problems using Python.
2. Write clean, readable, and well-structured Python programs following standard coding practices.
3. Use Python data types, operators, variables, input/output operations, and expressions effectively.
4. Implement decision-making and looping constructs to build dynamic and interactive applications.
5. Work confidently with Python collections, including Lists, Tuples, Sets, and Dictionaries, to store, organize, and manipulate data.
6. Perform string manipulation, formatting, validation, and text processing using built-in Python string methods.
7. Develop reusable and modular programs using functions, including parameter passing, return values, recursion, lambda functions, and variable-length arguments.
8. Organize applications using Python modules and packages while utilizing commonly used built-in libraries.

9. Implement robust error handling using exception handling techniques to develop reliable applications.
10. Read from and write to text, CSV, and JSON files for persistent data storage and retrieval.
11. Design and develop real-world console-based applications by integrating multiple Python concepts into a single solution.
12. Apply debugging and problem-solving techniques to identify, analyze, and resolve programming errors efficiently.
13. Develop algorithmic thinking and improve logical reasoning through hands-on coding exercises and problem-solving challenges.
14. Build a portfolio of practical projects, including management systems, utilities, games, and automation scripts using Core Python.
15. Gain a strong programming foundation required to progress confidently into Advanced Python, Web Development, Data Science, Artificial Intelligence, Automation, and Backend Development.

Skills Gained

By the end of the course, students will have hands-on experience in:

- Python Programming Fundamentals
- Problem Solving & Logical Thinking
- Data Structures (Lists, Tuples, Sets, Dictionaries)
- Functions & Modular Programming
- String Processing
- File Handling (Text, CSV, JSON)
- Exception Handling
- Working with Built-in Modules
- Console-Based Application Development
- Debugging & Code Optimization
- Software Development Best Practices

Scan to Start Your IT Career at JZ

